

UltimateEPM

Technical Documentation

Architecture, numerical methods, and library reference

UltimateEPM contributors

Contents

1	Introduction	3
1.1	Document organization	3
2	Current Scope	3
3	Analytical Monte Carlo Transport	3
3.1	Scope and Status	3
3.2	Semiclassical State	4
3.2.1	Carrier charge and numerical weight	4
3.2.2	Coordinate and unit conventions	4
3.3	Band and Valley Model	4
3.3.1	General ellipsoidal dispersion	4
3.3.2	Group velocity	5
3.3.3	Constant-energy state generation	5
3.3.4	Silicon electron valleys	5
3.3.5	Silicon hole bands	6
3.4	Initial Distribution	6
3.5	Deterministic Free Flight	6
3.6	Scattering Framework	6
3.6.1	Acoustic deformation-potential scattering	7
3.6.2	Electron intervalley phonon scattering	7
3.6.3	Hole optical phonon scattering	8
3.6.4	Ionized-impurity scattering	8
3.6.4.1	Empirical mobility model	8
3.6.4.2	Screened-Coulomb model	9
3.6.5	Impact ionization	9
3.7	Monte Carlo Time Integration	9
3.7.1	Fixed-time-step algorithm	9
3.7.2	Event-driven null-collision algorithm	10
3.7.3	Random-number streams and parallel execution	10
3.8	Device Transport Boundary Handling	10
3.9	Observables and Histories	11
3.10	Internal Transport Configuration	11
3.11	Device YAML Configuration	11
3.11.1	Input and run control	12
3.11.2	Time integration and self-consistency	12
3.11.3	Transport model	13
3.11.4	Contacts and initial particles	14
3.11.5	Two-dimensional geometry	14
3.11.6	Scheduled particle injection	14
3.11.7	Output control	15
3.11.8	Passive quench circuit	15
3.11.9	Avalanche and successful-quench detection	16
3.11.10	Complete generated schema	16
3.12	Implemented Invariants	17
3.13	Validation Coverage	17
3.14	Model Limitations	18
4	Documentation Conventions	18

1 Introduction

UltimateEPM is a C++20 scientific computing project for semiconductor simulation. Its capabilities include empirical pseudopotential band-structure calculations, Brillouin-zone processing, device meshing, Poisson solvers, and full-band and analytical Monte Carlo transport.

This document describes the internal architecture and numerical responsibilities of the libraries under `src/`. It may also include user-facing configuration references when they are needed to interpret the implemented numerical behavior. Generated API details remain the responsibility of Doxygen.

1.1 Document organization

Each source library has a dedicated chapter. A chapter should explain:

- the responsibility and boundaries of the library;
- its public types and principal entry points;
- the mathematical and numerical models it implements;
- its inputs, outputs, units, and invariants;
- its dependencies and interactions with other libraries;
- relevant validation and test coverage.

2 Current Scope

The present revision focuses on the analytical Monte Carlo (AMC) carrier-transport model and its device-simulation configuration. Device electrostatics, circuit coupling, avalanche detection, and quench detection are covered at the configuration and interaction level; complete numerical derivations and the other UltimateEPM libraries will be documented in later revisions.

3 Analytical Monte Carlo Transport

Source: `src/AMC`

CMake target: `lib_amc`

Alias: `uepm::amc`

Language: C++20

3.1 Scope and Status

The analytical Monte Carlo (AMC) model simulates semiclassical electron and hole transport in silicon. A numerical particle represents one or more physical carriers and evolves through alternating deterministic drift and stochastic scattering.

This chapter documents the transport model implemented by:

- `valley_model.hpp` and `valley_model.cpp`;
- `amc_material_model.hpp` and `amc_material_model.cpp`;
- `amc_scattering_model.hpp` and `amc_scattering_model.cpp`;
- `intervalley_phonon.hpp` and `intervalley_phonon.cpp`;
- `amc_transport_kernel.hpp` and `amc_transport_kernel.cpp`;
- `particle_amc.hpp` and `particle_amc.cpp`;
- the transport loops in `bulk_amc_simulation.cpp` and `device_amc_simulation.cpp`.

The current model is silicon-specific. Electrons use six anisotropic, nonparabolic Delta valleys. Holes use isotropic parabolic heavy-hole and light-hole bands. The transport kernel does not read these parameters from the project YAML files: they are currently compiled into `amc_material_model.cpp` and `intervalley_phonon.cpp`.

This chapter concentrates on the analytical transport model. It also documents the user-facing configuration of self-consistent Poisson coupling, induced-current output, passive quench-circuit coupling, avalanche detection, and successful-quench detection. Detailed derivations of those coupled models remain outside the present scope.

3.2 Semiclassical State

Each `particle_amc` owns a `particle_state`. The transport-relevant state is

$$S = (t, \mathbf{r}, \mathbf{k}_v, \mathbf{v}, E, \gamma, \nu),$$

where:

- t is the particle time in seconds;
- $\mathbf{r}$ is the position in device-mesh coordinates;
- $\mathbf{k}_v$ is the wave vector in the principal frame of the active valley or band, in m^{-1} ;
- $\mathbf{v}$ is the group velocity in the global device frame, in $\frac{\text{m}}{\text{s}}$;
- E is kinetic energy measured from the active valley or band edge, in eV;
- γ is the transformed kinetic energy, in eV;
- ν is the active electron-valley or hole-band index.

The particle also stores the local electric field, doping and impurity concentrations, its containing mesh element, its previous position, and a flag indicating collection by a contact.

3.2.1 Carrier charge and numerical weight

The signed carrier charge is

$$q_c = \begin{cases} -q_e & \text{for electrons} \\ +q_e & \text{for holes} \end{cases}$$

with $q_e = 1.602176634 \times 10^{-19}$ C.

The numerical particle weight $w > 0$ is independent of the microscopic transport trajectory. It scales charge and observables when one simulated particle represents several physical carriers. Scattering probabilities are not weight-dependent.

3.2.2 Coordinate and unit conventions

Three coordinate conventions meet in the transport code:

- the device mesh stores position in micrometres;
- the transport kernel stores velocity in $\frac{\text{m}}{\text{s}}$ and electric field in $\frac{\text{V}}{\text{m}}$;
- `particle_state::local_k` is expressed in the active valley's local principal-axis frame.

Consequently, the position update explicitly multiplies displacement in metres by 10^6 before adding it to the mesh position.

In a device simulation, the interpolated electric field is treated as $\frac{\text{V}}{\text{cm}}$ and multiplied by 10^2 before being passed to the transport kernel.

The impurity fields are stored in cm^{-3} . For device transport, the positive scattering-centre density is set to the absolute value of the interpolated net doping concentration:

$$N_I = |N_{\text{dop}}|.$$

3.3 Band and Valley Model

3.3.1 General ellipsoidal dispersion

Each `valley_model` defines a principal frame with transverse masses m_t and longitudinal mass m_l . In that frame,

$$\mathbf{k}_v = (k_{t1}, k_{t2}, k_l).$$

The transformed energy is the ellipsoidal quadratic form

$$\gamma(\mathbf{k}_v) = \frac{\hbar^2}{2q_e} \left(\frac{k_{t1}^2}{m_t} + \frac{k_{t2}^2}{m_t} + \frac{k_l^2}{m_l} \right),$$

where division by q_e , numerically equal to the joule-per-electronvolt conversion, expresses γ in eV.

For a parabolic band,

$$\gamma(E) = E.$$

For a Kane band,

$$\gamma(E) = E(1 + \alpha E),$$

where α is the nonparabolicity in eV^{-1} . The inverse relation used by the code is

$$E(\gamma) = \frac{\sqrt{1 + 4\alpha\gamma} - 1}{2\alpha}.$$

The parabolic relation is used when the band is explicitly parabolic or when $\alpha = 0$.

3.3.2 Group velocity

The group velocity in the valley frame follows from $\mathbf{v} = \hbar^{-1} \nabla_{\mathbf{k}} E$:

$$v_i = \frac{\hbar k_i}{m_i(1 + 2\alpha E)}.$$

For parabolic bands the denominator correction is one. The local velocity is rotated into the global device frame before it is stored in the particle.

Each valley owns an orthonormal rotation matrix R whose columns orient the local frame in global coordinates:

$$\mathbf{k}_v = R^T \mathbf{k}_g, \quad \mathbf{v}_g = R \mathbf{v}_v.$$

The constructor verifies $R^T R = I$ to a tolerance of 10^{-10} .

3.3.3 Constant-energy state generation

After a scattering event, the outgoing direction is sampled isotropically in the scaled ellipsoidal coordinate system. A unit vector $\mathbf{u}$ is drawn uniformly on the sphere and converted to a valley-frame wave vector:

$$\mathbf{k}_v = \frac{\sqrt{2q_e\gamma(E)}}{\hbar} (\sqrt{m_t}u_x, \sqrt{m_t}u_y, \sqrt{m_l}u_z).$$

This construction preserves the requested kinetic energy. It assumes isotropic angular redistribution in transformed momentum space. In particular, the screened-Coulomb rate is a momentum-relaxation rate, but the outgoing direction is still sampled isotropically rather than from the differential screened-Coulomb cross section.

3.3.4 Silicon electron valleys

Electrons occupy six equivalent Delta valleys:

Valleys	$\frac{m_t}{m_0}$	$\frac{m_l}{m_0}$	α	Axis
Δ_x^+, Δ_x^-	0.19	0.916	0.5eV^{-1}	x
Δ_y^+, Δ_y^-	0.19	0.916	0.5eV^{-1}	y
Δ_z^+, Δ_z^-	0.19	0.916	0.5eV^{-1}	z

All six minima have zero energy offset in the implemented model. The sign in the valley name identifies the opposite member of an axis pair, but both members currently use the same mass tensor and rotation matrix. The valley centre in the full Brillouin zone is not added to `local_k`; transport evolves only the wave vector relative to the selected minimum.

The conductivity and density-of-states masses exposed by `valley_model` are

$$m_c = \frac{3}{\frac{1}{m_l} + \frac{2}{m_t}}, \quad m_d = (m_l m_t^2)^{\frac{1}{3}}.$$

3.3.5 Silicon hole bands

Holes use two isotropic, parabolic bands:

Band	$\frac{m_t}{m_0}$	$\frac{m_l}{m_0}$	α
Heavy hole	0.87	0.87	0
Light hole	0.24	0.24	0

Both band edges have zero offset. Warping, split-off holes, anisotropy, and nonparabolicity are therefore omitted from the current hole model.

3.4 Initial Distribution

Particle initialization first selects a valley or band externally. Bulk and device simulations distribute initial particles cyclically over the available indices.

The initial kinetic energy is sampled from a gamma distribution with shape $\frac{3}{2}$ and scale $k_B T$:

$$p(E) \propto E^{\frac{1}{2}} \exp\left(-\frac{E}{k_B T}\right).$$

This is the three-dimensional Maxwell-Boltzmann energy distribution for a parabolic density of states. The same sampler is also used for Kane electron valleys; the nonparabolic density-of-states Jacobian is not included in the initial-energy distribution.

An isotropic transformed-space direction is then drawn and used to construct `local_k`, γ , kinetic energy, and global velocity consistently.

3.5 Deterministic Free Flight

Between scattering events, the semiclassical acceleration theorem is

$$\hbar \frac{d\mathbf{k}_g}{dt} = q_c \mathbf{F}_g.$$

The kernel rotates the electric field into the active valley frame and applies the exact constant-field wave-vector increment over a step Δt :

$$\mathbf{k}_v^{n+1} = \mathbf{k}_v^n + \frac{q_c}{\hbar} R^T \mathbf{F}_g \Delta t.$$

It then recomputes γ , E , and velocity from the updated wave vector. Position is advanced with the trapezoidal velocity:

$$\mathbf{r}^{n+1} = \mathbf{r}^n + \frac{\mathbf{v}^n + \mathbf{v}^{n+1}}{2} \Delta t.$$

The displacement is converted from metres to micrometres at this point. Particle time is incremented by Δt .

The field is assumed constant during one call to `drift_particle`. In device transport it is interpolated once at the beginning of the global time step. The implementation does not locate an exact mesh-face or contact crossing during the drift.

3.6 Scattering Framework

At a particle state (ν, E) , the kernel constructs a finite list of allowed channels j . Each channel contains:

- a physical mechanism;
- a rate Γ_j in s^{-1} ;
- the prescribed final kinetic energy;
- a destination band when applicable;
- an intervalley phonon branch and absorption/emission tag when applicable.

The total physical rate is

$$\Gamma_{\text{tot}(\nu, E)} = \sum_j \Gamma_{j(\nu, E)}.$$

Given that a real event occurs, channel selection uses the normalized rates:

$$P(j \mid \text{event}) = \frac{\Gamma_j}{\Gamma_{\text{tot}}}.$$

The channel container has a compile-time capacity of 16. The current electron model can populate acoustic, ten intervalley absorption/emission channels, impurity, and impact-ionization channels, which fits this capacity.

3.6.1 Acoustic deformation-potential scattering

Acoustic scattering is treated as elastic and equiprobable in angle. For electrons, the implemented rate can be written

$$\Gamma_{\text{ac}(E)} = \frac{\sqrt{2}k_B T m_d^{\frac{3}{2}} D_{\text{ac}}^2}{\pi \hbar^4 \rho u_{\text{avg}}^2} \sqrt{\gamma_{J(E)}} (1 + 2\alpha_J E_J),$$

with:

- silicon mass density $\rho = 2329 \frac{\text{kg}}{\text{m}^3}$;
- longitudinal sound speed $u_l = 9.0 \times 10^3 \frac{\text{m}}{\text{s}}$;
- transverse sound speed $u_t = 5.4 \times 10^3 \frac{\text{m}}{\text{s}}$;
- $u_{\text{avg}} = \frac{u_l + 2u_t}{3}$;
- acoustic deformation potential $D_{\text{ac}} = 6.6 \text{ eV}$;
- $\gamma_{J(E)} = E_J(1 + \alpha_J E_J)$.

For holes, the same functional dependence is used with:

- sound speed $u_s = 6.6 \times 10^3 \frac{\text{m}}{\text{s}}$;
- deformation potential $D_{\text{ac}} = 5.5 \text{ eV}$;
- overlap factor $\frac{1}{2}$.

The rate evaluation clamps energy to at least 10^{-9} eV before evaluating the square-root factor. Applying the event preserves kinetic energy, band or valley, and randomizes direction.

3.6.2 Electron intervalley phonon scattering

The electron model contains five silicon intervalley phonon branches:

Branch	Family	Order	$\hbar\omega$	Deformation potential	Z_f
g1_TA	g	First	11.4 “meV”	$D_1 = 3.0 \text{ eV}$	1
g2_LA	g	First	18.8 “meV”	$D_1 = 3.0 \text{ eV}$	1
g3_L0	g	Zeroth	63.2 “meV”	$D_0 = 3.4 \times 10^{10} \frac{\text{eV}}{\text{m}}$	1
f1_TA	f	First	21.9 “meV”	$D_1 = 3.0 \text{ eV}$	4
f2_LA	f	Zeroth	46.3 “meV”	$D_0 = 3.4 \times 10^{10} \frac{\text{eV}}{\text{m}}$	4

For a phonon energy $\hbar\omega$, the equilibrium Bose-Einstein occupation is

$$N_{\text{op}} = \frac{1}{\exp\left(\hbar \frac{\omega}{k_B T}\right) - 1}.$$

Absorption uses the factor N_{op} and

$$E_f = E_i + \hbar\omega.$$

Emission uses $N_{\text{op}} + 1$ and

$$E_f = E_i - \hbar\omega,$$

and is disabled when $E_f < 0$.

For a zeroth-order branch, the implemented rate is

$$\Gamma_0 = \frac{\sqrt{2}Z_fm_d^{\frac{3}{2}}D_0^2}{\pi\rho\hbar^2(\hbar\omega)}N_{\text{proc}}\sqrt{\gamma_{J(E_f)}}(1 + 2\alpha_J E_{f,J}),$$

where N_{proc} is the absorption or emission phonon factor.

For a first-order branch,

$$\Gamma_1 = \frac{\sqrt{2}Z_fm_d^{\frac{5}{2}}D_1^2}{\pi\rho\hbar^4(\hbar\omega)}N_{\text{proc}}\sqrt{\gamma_{J(E_f)}}(1 + 2\alpha_J E_{f,J})\left(\gamma_{J(E_f)} + \gamma_{J(E_i)}\right).$$

The branch multiplicity Z_f is included in the rate. At event application, a g process switches to the opposite valley on the same axis. An f process selects uniformly among the four valleys on the other two axes. The outgoing direction is randomized in the destination valley.

3.6.3 Hole optical phonon scattering

The hole model uses a single optical phonon energy of 63 meV and four transitions:

Transition	Destination	Overlap
heavy to heavy	heavy	0.5
heavy to light	light	1.0
light to heavy	heavy	1.0
light to light	light	0.5

All four use a deformation potential of $8.0 \times 10^{10} \frac{\text{eV}}{\text{m}}$. The rate has the same density-of-final-states form as the electron zeroth-order intervalley rate, multiplied by the transition overlap factor. Both absorption and emission are included, and the outgoing direction is randomized in the destination band.

3.6.4 Ionized-impurity scattering

Impurity scattering is optional. Bulk transport uses a configured uniform background density. Device transport uses the particle-local density interpolated from the mesh.

Two rate models are available.

3.6.4.1 Empirical mobility model

The Caughey-Thomas mobility is

$$\mu(N_I) = \mu_{\min} + \frac{\mu_0 - \mu_{\min}}{1 + \left(\frac{N_I}{N_{\text{ref}}}\right)^\beta}.$$

The implementation removes the lattice-limited contribution using Matthiessen's rule:

$$\frac{1}{\mu_I} = \frac{1}{\mu(N_I)} - \frac{1}{\mu_0},$$

then converts mobility to a scalar momentum-relaxation rate:

$$\Gamma_I = \frac{q_e}{m_d \mu_I}.$$

The compiled silicon parameters are:

Carrier	μ_0	$\mu_{\min}$	N_{ref}	β
Electron	$1417 \frac{\text{cm}^2}{\text{V s}}$	$52.2 \frac{\text{cm}^2}{\text{V s}}$	$9.68 \times 10^{16} \text{cm}^{-3}$	0.68
Hole	$470.5 \frac{\text{cm}^2}{\text{V s}}$	$44.9 \frac{\text{cm}^2}{\text{V s}}$	$2.23 \times 10^{17} \text{cm}^{-3}$	0.70

This model produces an energy-independent rate for a given carrier type and impurity density.

3.6.4.2 Screened-Coulomb model

The alternative model uses a Brooks-Herring-type screened-Coulomb momentum-relaxation rate. Screening is Debye-like:

$$q_s^2 = \frac{q_e^2 n_s}{\epsilon_s k_B T}, \quad \epsilon_s = 11.7 \epsilon_0.$$

With

$$k^2 = \frac{2m_d \gamma_{J(E)}}{\hbar^2}, \quad b = \frac{4k^2}{q_s^2},$$

the angular momentum-relaxation factor is

$$A(b) = \ln(1 + b) - \frac{b}{1 + b}.$$

The implementation forms the rate from $\frac{A(b)}{4k^4}$, impurity density, the silicon dielectric constant, and the Kane density-of-states correction.

In this function, electron nonparabolicity is fixed internally to 0.5eV^{-1} and hole nonparabolicity to zero, rather than read from the active `valley_model`. The screening density is currently set equal to the impurity density.

Both impurity models are applied as elastic, isotropic direction-randomizing events in the same band or valley.

3.6.5 Impact ionization

Impact ionization is optional and uses a threshold power law:

$$\Gamma_{\text{II}(E)} = \begin{cases} 0 & E \leq E_{\text{th}} \\ A \left(\frac{E - E_{\text{th}}}{E_{\text{th}}} \right)^p & E > E_{\text{th}}. \end{cases}$$

The compiled parameters are:

Carrier	E_{th}	A	p
Electron	1.12 “eV”	$1.1 \times 10^{14} \text{s}^{-1}$	2.5
Hole	1.49 “eV”	$1.4 \times 10^{12} \text{s}^{-1}$	3.4

When the event is applied, the primary carrier remains in its current band or valley, its kinetic energy is reduced by the threshold, and its direction is randomized:

$$E_f = \max(0, E_i - E_{\text{th}}).$$

In device transport, the event may additionally enqueue one electron and one hole at the event position, with the same numerical weight as the initiating particle. Those secondary particles receive independent thermal initial states. Energy and momentum are not partitioned among the three outgoing carriers, so this is a carrier-multiplication model rather than a microscopically energy-conserving final-state model.

3.7 Monte Carlo Time Integration

The code provides two distinct transport algorithms. They should not be described as numerically identical.

3.7.1 Fixed-time-step algorithm

`bulk_amc_simulation::run()` and device transport use a global step Δt :

1. Interpolate or assign the electric field.
2. Drift the particle for the full Δt .
3. Evaluate all rates at the post-drift state.
4. Compute the probability of at least one event, $P_{\text{sc}} = 1 - \exp(-\Gamma_{\text{tot}}\Delta t)$.
5. If an event is accepted, select and apply exactly one channel.

This method reproduces the no-event probability for a constant rate over the step, but permits at most one event. It is therefore a time-discretized approximation when $\Gamma_{\text{tot}}\Delta t$ is not small or when the field changes the rate significantly during the step. A practical accuracy condition is

$$\max(\Gamma_{\text{tot}}\Delta t) \ll 1.$$

The fixed-step path does not use the null-collision bound `gamma_max`.

3.7.2 Event-driven null-collision algorithm

`bulk_amc_simulation::run_self_scattering_emc()` implements a self-scattering, or null-collision, ensemble Monte Carlo algorithm.

For each band or valley ν , initialization samples the total rate on a uniform energy grid from zero to `m_max_energy_eV`. The majorant rate is

$$\Gamma_{\text{max,nu}} = s \max_{0 \leq E \leq E_{\text{max}}} \Gamma_{\text{tot}}(\nu, E),$$

where s is the configured safety factor. The default values are $E_{\text{max}} = 2$ eV, $s = 1.2$, and 1000 energy samples.

The candidate free-flight time is exponentially distributed:

$$\tau = -\frac{\ln U}{\Gamma_{\text{max,nu}}}, \quad U \in (0, 1).$$

After drifting for τ , the real-event acceptance probability is

$$P_{\text{real}} = \frac{\Gamma_{\text{tot}}(\nu, E)}{\Gamma_{\text{max,nu}}}.$$

If the candidate is rejected, a self-scattering event is recorded and the physical state is unchanged. If accepted, a real channel is selected from its normalized rate.

If an observed total rate exceeds its valley majorant, the thread-local kernel raises that majorant to the observed rate times the safety factor before acceptance. This prevents an acceptance probability above one for that candidate, but the already sampled flight time came from the old majorant. For unbiased null-collision transport, configuration should therefore ensure the precomputed majorant covers the physically reached energy range.

Because an intervalley event can change ν , the next free-flight sample uses the destination valley's majorant.

3.7.3 Random-number streams and parallel execution

The kernel uses `std::mt19937_64`. Constructors may seed from `std::random_device` or from an explicit 64-bit seed.

Parallel bulk and device runs create one transport kernel per OpenMP thread. Thread seeds are offset by 7919; electron and hole device streams receive adjacent base seeds. Results are reproducible for a fixed seed, thread count, particle ordering, and scheduling behavior, but changing the number of threads changes the random streams assigned to particles.

3.8 Device Transport Boundary Handling

After all particles drift for one device time step, the simulation updates mesh ownership and boundaries before scattering:

- a particle entering a contact is flagged and removed;
- a particle remaining inside its previous element keeps that element;
- otherwise, the device searches for a new containing element;

- if no element is found, or the new element is not silicon, the particle is returned to its pre-step position;
- reflection negates the complete velocity and local wave vector.

This is a full back-reflection, not specular reflection about a computed surface normal. The crossing time is not resolved, and the remaining fraction of the time step is not transported after reflection or contact collection.

The two-dimensional self-consistent implementation may additionally apply a periodic condition in the out-of-plane direction. The base three-dimensional device transport leaves this operation as a no-op.

3.9 Observables and Histories

A particle history records snapshots of time, position, local wave vector, velocity, kinetic energy, transformed energy, and band or valley index. It also counts:

- acoustic events;
- intervalley absorption;
- intervalley emission;
- impurity events;
- impact-ionization events;
- self-scattering events.

Bulk observables are accumulated after a configurable warm-up fraction. The code time-weights velocity and kinetic energy. The reported impact-ionization coefficient is based on

$$\alpha_{II} = \frac{\text{event rate per carrier}}{\text{average drift speed}},$$

with conversion from inverse metres to inverse centimetres.

The event-driven bulk path also exposes a per-particle raw estimate based on event count divided by displacement along x . That estimate assumes x is the drift direction and should not be used for arbitrary field orientation.

3.10 Internal Transport Configuration

The transport kernel is controlled by `amc_transport_config`:

Parameter	Default	Role
<code>m_carrier_type</code>	electron	Select electron valleys or hole bands.
<code>m_lattice_temperature</code>	300 K	Phonon occupation and initial thermal energy.
<code>m_max_energy_eV</code>	2 eV	Upper energy used to construct null-collision bounds.
<code>m_self_scattering_safety_factor</code>	1.2	Multiplier on sampled maximum rates.
<code>m_gamma_max_energy_samples</code>	1000	Uniform samples used for each majorant.
<code>m_background_impurity_density_cm⁻³</code>	0	Uniform bulk impurity density.
<code>m_enable_impact_ionization</code>	false	Enable threshold power-law channel.
<code>m_enable_impurity_scattering</code>	false	Enable an impurity channel.
<code>m_impurity_scattering_model</code>	mobility	Select empirical or screened-Coulomb rate.
<code>m_impurity_density_source</code>	background	Use configured or particle-local density.

`m_gamma_max_energy_samples` must be at least two in command-line option validation. The kernel's `initialize()` method itself assumes this condition when forming the uniform energy grid.

3.11 Device YAML Configuration

The `device_amc.epm` executable is configured primarily through YAML. The command-line interface intentionally contains only configuration-file selection, repeatable overrides, configuration generation, help, and version reporting.

A complete configuration template is generated with:

```
device_amc.epm --write-config config.yaml
```

A simulation is run with:

```
device_amc.epm --config config.yaml
```

Any scalar YAML value can be overridden without editing the file. Both `--set dotted.path=value` and `--set dotted.path value` are accepted, and `--set` may be repeated:

```
device_amc.epm --config config.yaml \  
  --set run.threads=8 \  
  --set contacts.cathode_voltage_V 30 \  
  --set simulation.final_time_s=2e-12
```

Configuration is resolved in this order:

1. compiled defaults establish the complete schema;
2. values present in the YAML file replace those defaults;
3. command-line `--set` assignments replace the resulting YAML values;
4. the fully resolved typed configuration is validated.

Unknown YAML keys, unknown override paths, mappings where scalar values are expected, invalid scalar types, and invalid enum strings are errors. This strict behavior is intentional: a misspelled key must not silently leave a default active.

The paths `input.device_mesh` and `input.material_file` are resolved relative to the directory containing the YAML file unless they are absolute. `run.output_directory` is not rebased against the YAML file; a relative output directory is interpreted from the process working directory.

3.11.1 Input and run control

YAML key	Default	Meaning
<code>input.device_mesh</code>	required	Gmsh device mesh and state file. The loaded mesh must be two- or three-dimensional. A relative path is based on the YAML directory.
<code>input.material_file</code>	project example	Material-parameter YAML used by the Poisson solver. A relative path is based on the YAML directory.
<code>input.material</code>	Si	Transport material symbol. The analytical device model currently accepts only silicon.
<code>run.name</code>	<code>self_consistent_and</code>	Human-readable simulation name stored in the run manifest and simulation options.
<code>run.output_directory</code>	empty	Directory for history, trajectory, visualization, and manifest outputs. If empty, the runner creates <code>self_consistent_amc_<mesh-stem></code> .
<code>run.threads</code>	1	Requested OpenMP transport-thread count. It must be strictly positive. Changing it also changes the assignment of random streams.
<code>run.seed</code>	0	Base integer seed used to initialize simulation random-number generators. Reproducibility also depends on thread count and scheduling.

3.11.2 Time integration and self-consistency

YAML key	Default	Meaning
<code>simulation.final_time_s</code>	10^{-12} s	Requested final physical simulation time. It must be strictly positive.

YAML key	Default	Meaning
<code>simulation.time_step_s</code>	10^{-15} s	Synchronized device Monte Carlo step. Drift, scattering, circuit evolution, and self-consistent bookkeeping use this temporal discretization. It must be strictly positive.
<code>simulation.temperature_K</code>	300 K	Lattice temperature used for phonon populations and thermal carrier initialization. It must be non-negative.
<code>simulation.max_particles</code>	10^9	Hard upper bound on the number of active numerical particles. The simulation rejects further growth rather than allowing unbounded avalanche multiplication. It must be non-zero.
<code>simulation.poisson_frequency</code>	10	Number of transport steps between self-consistent Poisson updates. It must be at least one.
<code>simulation.stop_when_no_electron</code>	true	Stop when no electron remains in the device, subject to pending scheduled injection and other continuation conditions. Set to false to continue until another stopping condition is reached.

The fixed-step accuracy discussion in the Monte Carlo time-integration section applies directly to `simulation.time_step_s`. In particular, it should be small relative to scattering and field-evolution time scales.

3.11.3 Transport model

YAML key	Default	Meaning
<code>transport.max_energy_eV</code>	10 eV	Maximum carrier energy sampled while constructing per-band or per-valley scattering-rate majorants. It must be strictly positive and should exceed energies reached during the run.
<code>transport.gamma_safety_factor</code>	1.2	Multiplicative safety margin applied to sampled maximum scattering rates. It must be strictly positive.
<code>transport.gamma_samples</code>	1000	Number of uniformly spaced energy samples used to construct each rate majorant. It must be at least two.
<code>transport.impact_ionization</code>	true	Include the impact-ionization scattering channel in rate evaluation.
<code>transport.particle_creation</code>	true	When an impact-ionization event occurs, create the secondary electron-hole pair. If false, impact-ionization events can still be computed and recorded but do not multiply carriers.
<code>transport.impurity_scattering</code>	false	Enable ionized-impurity scattering using local device doping as the scattering-centre density.
<code>transport.impurity_model</code>	mobility	Impurity-rate model. Accepted values are mobility for the empirical mobility-derived rate and screened-coulomb for the analytical screened-Coulomb rate.
<code>transport.impurity_screening</code>	debye	Screening approximation for the screened-Coulomb model. Accepted values are debye for analytic Debye screening and full for the finite-temperature numerical treatment. It has no effect when impurity scattering is disabled or the mobility model is selected.

3.11.4 Contacts and initial particles

YAML key	Default	Meaning
<code>contacts.anode_voltage_V</code>	0 V	Dirichlet voltage applied to the automatically created anode contact unless that contact is the dynamically biased quench-circuit node.
<code>contacts.cathode_voltage_V</code>	0 V	Dirichlet voltage applied to the automatically created cathode contact unless that contact is the dynamically biased quench-circuit node.
<code>particles.initial_electrons</code>	1	Number of explicit electrons created at <code>particles.initial_position</code> before transport starts.
<code>particles.initial_holes</code>	0	Number of explicit holes created at <code>particles.initial_position</code> before transport starts.
<code>particles.initial_position.x_um</code>	0 um	Initial mesh-coordinate x position of explicitly requested particles.
<code>particles.initial_position.y_um</code>	0 um	Initial mesh-coordinate y position of explicitly requested particles.
<code>particles.initial_position.z_um</code>	0 um	Initial mesh-coordinate z position of explicitly requested particles.
<code>particles.initialize_from_doping</code>	true	Also initialize numerical carriers from the mesh doping distribution.
<code>particles.initial_weight</code>	2	Numerical weight assigned to particles generated from the initial doping distribution. It must be positive.
<code>particles.contact_injection_weight</code>	2	Numerical weight assigned to carriers injected by contact charge reservoirs during self-consistent evolution. It must be positive.

The explicit counts and doping-based initialization are independent. Thus, a run may contain both the requested particles at the configured position and additional particles representing the initial dopant charge.

3.11.5 Two-dimensional geometry

YAML key	Default	Meaning
<code>geometry_2d.effective_depth_um</code>	1 um	Physical out-of-plane depth represented by a two-dimensional mesh. It scales integrated doping, deposited charge, and Ramo current. Ignored for a three-dimensional mesh.
<code>geometry_2d.particle_z_period_um</code>	1 um	Numerical periodic length used for particle z coordinates in a two-dimensional simulation. Ignored for a three-dimensional mesh.

The effective physical depth and numerical periodic length are separate concepts and need not be equal.

3.11.6 Scheduled particle injection

YAML key	Default	Meaning
<code>scheduled_injection.enabled</code>	false	Enable one scheduled particle injection during the simulation.
<code>scheduled_injection.time_s</code>	0 s	Physical time at which the particle is injected. It must be non-negative and no greater than <code>simulation.final_time_s</code> when injection is enabled.
<code>scheduled_injection.position.x_um</code>	0 um	Injection x position in device-mesh coordinates.

YAML key	Default	Meaning
<code>scheduled_injection.position.y</code>	um 0 um	Injection y position in device-mesh coordinates.
<code>scheduled_injection.position.z</code>	um 0 um	Injection z position in device-mesh coordinates.
<code>scheduled_injection.type</code>	electron	Injected carrier type. Accepted values are electron, e, hole, and h.
<code>scheduled_injection.weight</code>	1	Numerical carrier weight assigned to the injected particle. It must be positive when injection is enabled.

Only one scheduled injection is represented by the current schema. After it has been performed, its internal done state prevents reinjection.

3.11.7 Output control

YAML key	Default	Meaning
<code>output.keep_particle_history</code>	false	Store complete per-particle trajectories and export them after the run. This can consume substantial memory and storage for large avalanches.
<code>output.export_time_steps</code>	false	Periodically export particle and mesh snapshots for visualization.
<code>output.export_frequency</code>	100	Number of transport iterations between snapshot exports. It must be strictly positive, even when periodic export is disabled.

Every completed run writes `device_history.csv` and `simulation_manifest.txt`. The manifest records the command line, resolved simulation parameters, build information, input paths, output paths, and final observables. Trajectory and time-step directories are created only when the corresponding output mode requires them.

3.11.8 Passive quench circuit

YAML key	Default	Meaning
<code>quench_circuit.enabled</code>	true	Couple the selected device contact to the passive series-resistor and parallel-capacitance quench model.
<code>quench_circuit.resistance_ohm</code>	1 ohm	Quench resistance R . Circuit validation requires a strictly positive, finite value.
<code>quench_circuit.capacitance_F</code>	1 F	Device or quench-node capacitance C . Circuit validation requires a strictly positive value.
<code>quench_circuit.biased_contact</code>	cathode	Contact whose voltage is evolved by the circuit. Accepted values are anode and cathode.
<code>quench_circuit.ramo_current_sign</code>	-1	Non-zero sign and scale converting signed Ramo current into current drawn from the biased circuit node. Reverse it when the mesh/contact convention gives the opposite physical current polarity.
<code>quench_circuit.background_ramo_current_A</code>	0 A	Baseline current subtracted from the measured Ramo-current signal before self-consistent circuit and stopping logic use it.

The circuit supply voltage and initial device voltage are derived from the configured voltage of `quench_circuit.biased_contact`. For example, with the default cathode selection, both are initialized from `contacts.cathode_voltage_V`; they are not independent YAML parameters.

3.11.9 Avalanche and successful-quench detection

YAML key	Default	Meaning
<code>avalanche_detection.voltage_drop_V</code>	1 V	Absolute quench-circuit voltage drop required to mark avalanche onset. It must be positive and finite.
<code>quench_detection.high_field_V_per_cm</code>	10^5 V/cm	A particle at or above this electric-field magnitude resets the quiet interval used for successful-quench detection. It must be positive and finite.
<code>quench_detection.quiet_time_s</code>	10^{-11} s	Required time after avalanche without high-field particles or new impact-ionization events before quenching is declared successful. It must be positive and finite.

Detection results, event times, voltage drop, and the interval from avalanche to successful quench are written to the simulation manifest when available.

3.11.10 Complete generated schema

The generated file contains every currently accepted key with its compiled default. It is the preferred starting point for a new run:

```
input:
  device_mesh: device.msh
  material_file: /path/to/examples/materials/materials.yaml
  material: Si
run:
  name: self_consistent_amc
  output_directory: ""
  threads: 1
  seed: 0
simulation:
  final_time_s: 1e-12
  time_step_s: 1e-15
  temperature_K: 300
  max_particles: 1000000000
  poisson_frequency: 10
  stop_when_no_electrons: true
transport:
  max_energy_eV: 10
  gamma_safety_factor: 1.2
  gamma_samples: 1000
  impact_ionization: true
  particle_creation: true
  impurity_scattering: false
  impurity_model: mobility
  impurity_screening: debye
contacts:
  anode_voltage_V: 0
  cathode_voltage_V: 0
particles:
  initial_electrons: 1
  initial_holes: 0
  initial_position:
    x_um: 0
    y_um: 0
    z_um: 0
  initialize_from_doping: true
  initial_weight: 2
  contact_injection_weight: 2
```

```

geometry_2d:
  effective_depth_um: 1
  particle_z_period_um: 1
scheduled_injection:
  enabled: false
  time_s: 0
  position:
    x_um: 0
    y_um: 0
    z_um: 0
  type: electron
  weight: 1
output:
  keep_particle_history: false
  export_time_steps: false
  export_frequency: 100
quench_circuit:
  enabled: true
  resistance_ohm: 1
  capacitance_F: 1
  biased_contact: cathode
  ramo_current_sign: -1
  background_ramo_current_A: 0
avalanche_detection:
  voltage_drop_V: 1
quench_detection:
  high_field_V_per_cm: 100000
  quiet_time_s: 1e-11

```

3.12 Implemented Invariants

The transport classes enforce several local invariants:

- particle weight is strictly positive;
- kinetic energy and γ cannot be set negative through their setters;
- drift and scattering time increments cannot be negative;
- effective masses are positive;
- nonparabolicity is non-negative;
- valley degeneracy is at least one;
- valley rotations are orthonormal;
- final phonon-emission energy cannot be negative;
- channel rates passed to event application cannot be negative.

Direct public access through `particle_amc::state()` can bypass some particle setter checks. Simulation code must preserve state consistency:

$$\gamma = \gamma(\mathbf{k}_v), \quad E = E(\gamma), \quad \mathbf{v} = \mathbf{v}(\mathbf{k}_v, E).$$

The transport kernel restores these relations after initialization, drift, and every real scattering event.

3.13 Validation Coverage

`tests/amc/test_amc_transport_kernel.cpp` currently verifies:

- the sum of constructed channel rates equals the reported total rate;
- a per-valley null-collision majorant is positive and no larger than the global maximum;
- a zero-duration fixed-step scattering call leaves the particle unchanged.

These tests exercise basic bookkeeping but do not yet validate the physical rate equations or transport observables.

Recommended model-validation cases are:

- energy and velocity consistency for every valley orientation;

- Maxwell-Boltzmann equilibrium at zero field with impact ionization disabled;
- detailed-balance ratios for phonon absorption and emission;
- low-field electron and hole mobility versus temperature and doping;
- velocity-field curves for bulk silicon;
- valley populations under fields along the principal crystal axes;
- convergence with fixed time step;
- convergence with null-collision energy-grid resolution and safety factor;
- impact-ionization coefficient versus electric field;
- statistical reproducibility and confidence intervals across independent random seeds.

3.14 Model Limitations

The following limitations are part of the current implementation and should be kept visible when interpreting results:

- only silicon is supported;
- material and scattering parameters are compiled into C++ sources;
- electron transport uses six analytical Delta valleys with no explicit Brillouin-zone valley centres;
- hole transport uses isotropic parabolic heavy- and light-hole bands;
- the initial Kane-electron energy sampler uses the parabolic Maxwell-Boltzmann density of states;
- all real scattering events use isotropic outgoing directions in transformed momentum space;
- acoustic scattering is elastic and uses an equipartition approximation;
- no carrier-carrier scattering or Pauli blocking is included;
- the screened-Coulomb differential angular distribution is not sampled;
- impact ionization does not conserve the complete outgoing carrier energy and momentum microscopically;
- device fields are constant during a global transport step;
- device boundary crossing times and surface normals are not resolved;
- the fixed-step algorithm permits at most one event per step;
- null-collision accuracy depends on the configured energy range containing all reached carrier energies.

These limitations do not make the model unusable, but they define the level of physical fidelity that validation must establish for each intended operating regime.

4 Documentation Conventions

Physical quantities must be documented with their units. When a quantity is stored in a unit different from the one used by an equation or input file, both the storage unit and the conversion point must be stated.

Public type and function names are written as code, for example `uepm::poisson`. File paths are relative to the repository root. Mathematical symbols should be defined on first use and used consistently throughout a chapter.

Statements about implemented behavior should be traceable to source code or a test. Planned behavior must be explicitly labeled as such.